

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЖИТОМИРСЬКА ПОЛІТЕХНІКА
Кафедра ММСА

ЛАБОРАТОРНА РОБОТА № 3
з дисципліни «Моделювання та аналіз програмного забезпечення»

**ПОБУДОВА ДІАГРАМ ОБ'ЄКТІВ,
КОМПОНЕНТІВ ТА РОЗГОРТАННЯ**

Виконав: студент групи ВТ-23-2
Камінський Олексій Дмитрович

Перевірив: _____

Житомир – 2026

Мета

Опанувати три структурні діаграми UML (Unified Modeling Language — уніфікована мова моделювання), що доповнюють діаграму класів Лабораторної роботи №2: діаграму об'єктів (Object Diagram) — конкретний «знімок» системи у фіксований момент часу; діаграму компонентів (Component Diagram) — модульну архітектуру системи з наданими й необхідними інтерфейсами; діаграму розгортання (Deployment Diagram) — фізичне розміщення артефактів на вузлах інфраструктури. Додаткова мета — забезпечити наскрізну узгодженість моделі: кожен компонент має походити від класів Лабораторної роботи №2, а кожна зовнішня система — від актора Лабораторної роботи №1.

Предметна область та сама, що й у Лабораторних роботах №1 і №2: веб-сервіс завантаження та читання книг «Полиця» — спрощений аналог MyBook і Bookmate. Користувач реєструється, переглядає каталог, завантажує власні книги у форматах EPUB або PDF, читає їх у вбудованій читалці зі збереженням прогресу й закладок, веде власні колекції та оформлює підписку для доступу до платного каталогу. Модератор перевіряє завантажений користувачами контент; оплату обробляє зовнішня платіжна система, файли книг зберігаються в зовнішньому об'єктному сховищі, вхід можливий через Google OAuth, а переклад виділеного слова виконує зовнішній словниковий сервіс.

Розділ 1. Інструмент і порядок рендерингу діаграм

Усі три діаграми побудовано в текстовому CASE-інструменті PlantUML — тому самому, що й у Лабораторних роботах №1 і №2. Вибір пояснювався в Лабораторній роботі №1: текстове джерело версіонується в Git, а діаграму можна перебудувати однією командою. Для цієї роботи така властивість виявилася критичною: обидві «великі» діаграми (компонентів і розгортання) довелося кілька разів переверстувати, і кожна ітерація коштувала лише одного запуску рендера.

Версії інструментів, на яких виконано роботу:

```
$ plantuml -version
PlantUML version 1.2026.8 / 149874a (2026-09-05 15:59:17 UTC)
Java Runtime: OpenJDK Runtime Environment
Default Encoding: UTF-8
PLANTUML_LIMIT_SIZE: 4096
GraphViz: dot - graphviz version 16.1.0 (20260904.0139)
Installation seems OK. File generation OK
```

Для кожного файлу спершу виконувалася перевірка синтаксису без побудови зображення, а потім рендеринг у два формати — SVG (векторний, для перегляду) і PNG (растровий, для вставляння у звіт):

```
$ plantuml -checkonly objects.puml && echo $?  
0  
$ plantuml -tsvg objects.puml && plantuml -tpng objects.puml  
$ plantuml -checkonly components.puml && echo $?  
0  
$ plantuml -tsvg components.puml && plantuml -tpng components.puml  
$ plantuml -checkonly deployment.puml && echo $?  
0  
$ plantuml -tsvg deployment.puml && plantuml -tpng deployment.puml
```

Оскільки всі підписи українські, у кожен файл додано рядок `skinparam defaultFontName "Helvetica"` — без нього PlantUML обирає шрифт, у якому кирилиця на деяких системах відображається прямокутниками.

Фактичний результат рендерингу:

Файл	Розмір <code>.puml`</code>	Розмір <code>.svg`</code>	Розмір <code>.png`</code>	Роздільність <code>.png`</code>
<code>`objects`</code>	5,5 КБ	21,0 КБ	66,3 КБ	1310 × 986
<code>`components`</code>	6,1 КБ	50,3 КБ	214,6 КБ	1482 × 1456
<code>`deployment`</code>	7,1 КБ	60,7 КБ	125,3 КБ	2745 × 1753

Окремо варто зафіксувати дві реальні проблеми розкладки, бо вони пояснюють наявність у файлах службових рядків `-[hidden]down-`.

Проблема 1 — діаграма компонентів розповзлася в один рядок. Перший рендер дав зображення 2899 × 433 пікселі: Graphviz вишикував усі 18 компонентів в один горизонтальний ряд, а підписи інтерфейсів («ІБібліотека», «ІЗавантаження», «ІАвтентифікація») накладалися один на одного, бо кружечок інтерфейсу — дуже малий вузол, і Graphviz не враховує ширину його підпису при розрахунку відстаней. Збільшення `nodesep` до 75 прибрало накладання, але розтягнуло діаграму до 2519 пікселів завширшки. Остаточне рішення — директива `left to right direction`: у режимі «зліва направо» вузли одного рангу складаються вертикально, тож підписи інтерфейсів опиняються кожен на своєму рядку й не можуть перетнутися в принципі.

Проблема 2 — діаграма розгортання вперлася в ліміт розміру. Перший рендер дав 4096 × 605 — рівно ліміт `PLANTUML_LIMIT_SIZE`, тобто зображення було обрізане. Причина технічна: кожен вузол із вкладеним вмістом Graphviz перетворює на кластер, а кластери між собою він не ранжує — тому всі 11 вузлів опинилися на одному рівні. Обхід: підказки розкладки задано не між вузлами, а між артефактами всередині них (`ArtNginx -[hidden]down- ArtGateway`), бо вузли

всередині різних кластерів Graphviz ранжує нормально. Після цього діаграма отримала правильну пошарову структуру і вкладається в 2745 пікселів завширшки. Такі самі приховані зв'язки застосовано й у Лабораторній роботі №2 з тієї ж причини, тож прийом уже перевірений.

Проблема 3 — діаграма об'єктів розтягнулася в один рядок через повні хеші.

Після того, як на знімок додано обов'язкові об'єкти метадані і третій об'єкт файлкиги (обґрунтування — у розділі 2.1), а значення хешSHA256 показано повністю, усіма 64 шістнадцятковими символами, перший рендер дав 2752×696 : Graphviz вишикував усі десять об'єктів в один горизонтальний ряд, бо широкі прямокутники «не вміщалися» в жодну іншу розкладку. Три приховані зв'язки (ометаданікобзар -[hidden]down- офайлерув і ще два) опустили метадані й файли на нижні рівні, після чого діаграма стиснулася до 1310×986 і стала читабельною.

Важливо підкреслити: рядки -[hidden]down- **не є елементами UML**. Це виключно підказки для Graphviz, вони не відображаються на рисунку і не несуть жодного модельного змісту. У файлах вони винесені в окремий блок із відповідним коментарем, щоб не плутати їх з реальними зв'язками.

Висновок: усі три діаграми успішно пройшли перевірку синтаксису (plantuml -checkonly повернув код 0) і відрендерені у два формати; жодне зображення не обрізане лімітом розміру 4096 пікселів.

Розділ 2. Діаграма об'єктів: знімок стану системи

2.1. Який фрагмент діаграми класів ілюструє цей знімок

Діаграма об'єктів — це не самостійна модель, а ілюстрація до вже побудованої діаграми класів. Тому спершу слід чітко назвати, який саме її фрагмент ми «фотографуємо».

Знімок охоплює **шість класів** Лабораторної роботи №2, що утворюють замкнений ланцюг від читача до файла книги: Читач → Колекція → Книга (нащадок Публікація) → Файлкиги та Метадані, і паралельну гілку Читач → ПрогресЧитання → книга. Цей фрагмент обрано свідомо, бо він єдиний у моделі, де одночасно присутні всі три «структурні» типи відношень діаграми класів — композиція, агрегація та звичайна асоціація, — а отже на одному знімку можна перевірити коректність усіх трьох.

Завдання пропонує обрати 4–5 класів, а тут їх шість, і це перевищення продиктоване не бажанням «зробити більше», а вимогою коректності. У Лабораторній роботі №2 клас публікація зв'язаний двома **обов'язковими**

композиціями: Публікація "1" *-- "1" Метадані : описується і Публікація "1" *-- "1..*" ФайлКниги : містить. Нижня межа в обох випадках не нульова. Отже знімок, на якому в книги немає об'єкта Метадані або немає жодного об'єкта ФайлКниги, зображує **неможливий стан системи**: він суперечить власній діаграмі класів, а значить нічого не перевіряє, а лише створює враження перевірки. Саме тому до знімка входять об'єкти метадані_кобзар і метадані_тигролови, а книга «Тигролови» отримала власний файл тигролови_erub. Шостий клас тут не прикраса — без нього головна цінність діаграми об'єктів зникає.

Момент часу знімка — **7 жовтня 2026 року, 08:30**: читач Олексій Камінський відкриває сервіс, маючи одну власну полицю з двома книгами й незавершене читання «Кобзаря».

Об'єкт на знімку	Клас із ЛР2	Що ілюструє
`читач_kaminskyi`	`Читач` (нащадок `КористувачСистеми`)	Атрибути обох рівнів ієрархії зібрані в одному об'єкті
`полиця_класика`	`Колекція`	Частина композиції `Читач` ♦ `Колекція`
`кобзар`	`Книга` (нащадок `Публікація`)	Безплатна позиція публічного каталогу
`тигролови`	`Книга`	Платна позиція — показує, що `платна` є атрибутом, а не окремим класом
`метадані_кобзар`	`Метадані`	Обов'язкова частина композиції `Публікація` ♦ `Метадані` (`1` до `1`)
`метадані_тигролови`	`Метадані`	Та сама обов'язкова композиція для другої книги
`кобзар_erub`	`ФайлКниги`	Формат EPUB тієї самої книги
`кобзар_pdf`	`ФайлКниги`	Формат PDF тієї самої книги — «розгорнута» множинність `1..*`
`тигролови_erub`	`ФайлКниги`	Нижня межа тієї самої множинності: рівно один файл
`прогрес_кобзар`	`ПрогресЧитання`	Позиція читача в конкретній публікації

Три рішення в оформленні заслуговують окремого пояснення.

Перше: `кобзар : Книга`, а не `кобзар : Публікація`. Клас публікація у Лабораторній роботі №2 оголошено абстрактним, а абстрактний клас за визначенням не має власних екземплярів. Тому об'єкт на знімку може бути екземпляром лише конкретного нащадка — книга або документ. При цьому в

прямокутнику об'єкта показано значення **всіх** атрибутів, зокрема успадкованих від Публікація (id, назва, мова, датаДодавання, статусМодерації): на знімку реального об'єкта спадкування вже «сплющене», бо фізично це один об'єкт з одним набором полів.

Друге: на лінках немає множинності й немає ромбів. Згідно з таблицею розділу 1.1 методички, лінк — це конкретний екземпляр асоціації, і множинність на ньому не вказується, бо вона вже розгорнута в конкретну кількість об'єктів. Саме тому на знімку навмисно показано **два** об'єкти ФайлКниги для однієї книги (ілюстрація множинності 1..* композиції Публікація ♦ ФайлКниги) і **дві** книги в одній колекції (ілюстрація множинності 0..* агрегації Колекція ◇ Публікація). Числа «1» і «0..*» на самій діаграмі об'єктів були б дублюванням і помилкою нотації.

Третє: `хешПароля = null` і повні 64-символьні хеші. Якщо в прямокутнику заявлено значення **всіх** атрибутів, то жоден атрибут не можна мовчки пропустити — навіть той, що на перший погляд «не застосовується». Читач читач_kaminskyi входить у сервіс виключно через Google OAuth (у нього заповнено googleId), тому успадкований від КористувачСистеми атрибут хешПароля має значення null — і саме це значення показано. Пропустити рядок було б двозначно: читач не зрозумів би, чи атрибута немає в класі, чи його просто забули намалювати. З тієї самої причини значення хешSHA256 подано повністю, усіма 64 шістнадцятковими символами, а не у вигляді 9f2b4c8d3a...e71a: багатокрапка — це заповнювач, тоді як п. 2 Частини 1 завдання вимагає конкретних правдоподібних значень. Ціною стала ширина прямокутників, яку довелося компенсувати розкладкою (Проблема 3 розділу 1).

2.2. Код діаграми об'єктів

Файл objects.puml:

```
@startuml
' Лабораторна робота №3. Частина 1. Діаграма об'єктів
' Предметна область: веб-сервіс завантаження та читання книг «Полиця»
' Камінський Олексій Дмитрович, група ВТ-23-2
' Знімок стану системи на 2026-10-07 08:30
' Знімок є ДОПУСТИМИМ екземпляром діаграми класів ЛР2: виконано обидві
' обов'язкові композиції Публікація♦Метадані (1..1) і Публікація♦ФайлКниги (1..*).

skinparam defaultFontName "Helvetica"
skinparam shadowing false
skinparam ArrowFontSize 11
skinparam nodesep 30
skinparam ranksep 55
skinparam object {
    BackgroundColor #FEFEFE
    BorderColor #33668E
    ArrowColor #33668E
}
```

```
object "<u>читач_kaminskyi : Читач</u>" as оЧитач {
  id = "7e3a1c2f-9b44-4d81-a6e0-2f5c8b013d77"
  email = "o.kaminskyi@example.com"
  хешПароля = null
  статусАкаунта = "активний"
  датаРеєстрації = "2026-02-14 10:22"
  ім'я = "Олексій"
  моваІнтерфейсу = "uk"
  лімітЗавантажень = 20
  googleId = "109482733015627740391"
}

object "<u>полиця_класика : Колекція</u>" as оКолекція {
  назва = "Українська класика"
  приватна = false
  датаСтворення = "2026-03-02"
}

object "<u>кобзар : Книга</u>" as оКобзар {
  id = "b1d9f0a4-5c27-4e6b-8f13-77ac9e2d4410"
  назва = "Кобзар"
  мова = "uk"
  датаДодавання = "2026-01-20 09:05"
  статусМодерації = "схвалено"
  автор = "Тарас Шевченко"
  isbn = "978-966-03-7891-2"
  жанр = "Поезія"
  платна = false
}

object "<u>тигрови : Книга</u>" as оТигрови {
  id = "4af07b63-18ce-42aa-9d05-6b1e30f7c982"
  назва = "Тигрови"
  мова = "uk"
  датаДодавання = "2026-02-28 18:41"
  статусМодерації = "схвалено"
  автор = "Іван Багряний"
  isbn = "978-617-12-4455-0"
  жанр = "Пригодницький роман"
  платна = true
}

object "<u>метадані_кобзар : Метадані</u>" as оМетаданіКобзар {
  автор = "Тарас Шевченко"
  видавництво = "А-ба-ба-га-ла-ма-га"
  рікВидання = 2018
  обкладинкаURL = "https://cdn.polytsia.ua/covers/b1d9f0a4.jpg"
}

object "<u>метадані_тигрови : Метадані</u>" as оМетаданіТигрови {
  автор = "Іван Багряний"
  видавництво = "Книжковий клуб «Клуб сімейного дозвілля»"
  рікВидання = 2021
  обкладинкаURL = "https://cdn.polytsia.ua/covers/4af07b63.jpg"
}

object "<u>кобзар_epub : ФайлКниги</u>" as оФайлEPUB {
  ключОб'єкта = "books/b1d9f0a4/kobzar.epub"
  формат = "EPUB"
  розмірБайт = 2418544
  хешSHA256 = "1c5d32ecdb6792064723f2f6a61c835c54b0068a205c48b8524fe69ca32c64f5"
}
```

```

object "<u>кобзар_pdf : ФайлКниги</u>" as оФайлPDF {
    ключОб'єкта = "books/b1d9f0a4/kobzar.pdf"
    формат = "PDF"
    розмірБайт = 11735208
    хешSHA256 = "ac0131dad44f7a041dd61b7f187cfada44a105d420b2cb6da278dc426bf89ff4"
}

object "<u>тигровлови_epub : ФайлКниги</u>" as оФайлТигровлови {
    ключОб'єкта = "books/4af07b63/tyhrolovy.epub"
    формат = "EPUB"
    розмірБайт = 3907126
    хешSHA256 = "937d9253789d9eb92587eb0385a26e0a6d6a9ab83ed5c135cac22a276f93e8f2"
}

object "<u>прогрес_кобзар : ПрогресЧитання</u>" as оПрогрес {
    позиціяСторінки = 148
    відсотокПрочитаного = 37.2
    оновлено0 = "2026-10-06 22:14"
    пристрій = "iPhone 15, Safari"
}

' ===== ЛІНКИ (конкретні екземпляри асоціацій ЛР2) =====
' Множинність на лінках не вказується: вона вже «розгорнута»
' у конкретну кількість об'єктів на цьому знімку.
очитач -- оКолекція : веде
очитач -- оПрогрес : накопичує
оКолекція -- оКобзар : містить
оКолекція -- оТигровлови : містить
оКобзар -- оМетаданіКобзар : описується
оТигровлови -- оМетаданіТигровлови : описується
оКобзар -- оФайлEPUB : містить
оКобзар -- оФайлPDF : містить
оТигровлови -- оФайлТигровлови : містить
оПрогрес -- оКобзар : фіксує позицію в

' ===== ПРИХОВАНІ ЗВ'ЯЗКИ ДЛЯ КОМПАКТНОЇ РОЗКЛАДКИ =====
' (не є елементами UML – лише підказки розкладки для Graphviz:
' без них усі десять об'єктів вишиковуються в один рядок
' завширшки 2752 пікселі через довгі 64-символьні хеші SHA-256).
оМетаданіКобзар -[hidden]down- оФайлEPUB
оФайлEPUB -[hidden]down- оФайлPDF
оМетаданіТигровлови -[hidden]down- оФайлТигровлови
@enduml

```

2.3. Рисунок

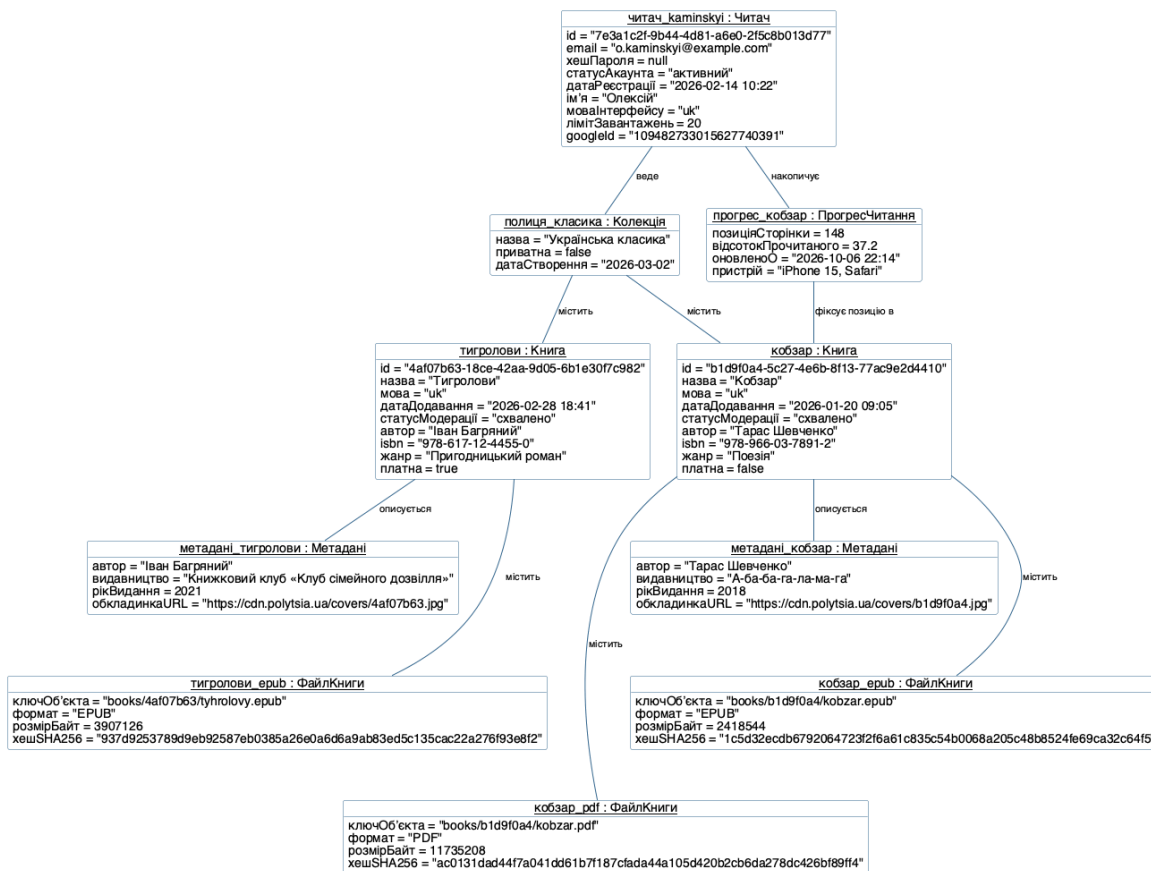


Рисунок 1 — Діаграма об'єктів: знімок стану сервісу «Полиця» на 2026-10-07 08:30

2.4. Відповідність лінків асоціаціям діаграми класів

Головна перевірка коректності діаграми об'єктів — кожен лінк має бути екземпляром якоїсь асоціації діаграми класів. Лінка «нізвідки» бути не може. Нижче наведено повну звірку всіх десяти лінків знімка з відношеннями Лабораторної роботи №2.

Лінк на знімку	Відношення в ЛР2	Тип	Множинність у ЛР2	Як вона розгорнута на знімку
`читач_kaminskyi — полиця_класика`	`Читач *-- Колекція : веде`	Композиція	`1` до `0..*`	Один читач, одна полиця (припустимий окремий випадок `0..*`)
`читач_kaminskyi — прогрес_кобзар`	`Читач *-- ПрогресЧитання : накопичує`	Композиція	`1` до `0..*`	Один читач, один запис прогресу
`полиця_класика — кобзар`	`Колекція о-- Публікація : містить`	Агрегація	`0..*` до `0..*`	Одна полиця, дві книги — звідси два окремі лінки
`полиця_класика — тигролови`	`Колекція о-- Публікація : містить`	Агрегація	`0..*` до `0..*`	—
`кобзар — метадані_кобзар`	`Публікація *-- Метадані : описується`	Композиція	`1` до `1`	Обов'язковий лінк: рівно один об'єкт

				Метадані на книгу
тигрови — метадані_тигрови`	Публікація *-- Метадані : описується`	Композиція	1` до 1`	—
кобзар — кобзар_epub`	Публікація *-- ФайлКниги : містить`	Композиція	1` до 1..*`	Одна книга, два файли — саме це й доводить, що 1..*` не помилка
кобзар — кобзар_pdf`	Публікація *-- ФайлКниги : містить`	Композиція	1` до 1..*`	—
тигрови — тигрови_epub`	Публікація *-- ФайлКниги : містить`	Композиція	1` до 1..*`	Нижня межа тієї самої множинності: рівно один файл
прогрес_кобзар — кобзар`	ПрогресЧитання -- Публікація : фіксує позицію в`	Асоціація	0..*` до 1`	Один запис прогресу вказує рівно на одну публікацію

Знімок не лише ілюструє модель, а й справді **перевіряє** її — у цьому і полягає практична цінність діаграми об'єктів. Перша перевірка формальна і найсуворіша: кожен об'єкт знімка задовольняє **нижні межі** всіх обов'язкових відношень діаграми класів. Для класу книга таких меж дві — рівно один об'єкт Метадані і щонайменше один об'єкт ФайлКниги, — і обидві виконано для обох книг. Саме ця перевірка і змусила додати до знімка три об'єкти, яких у першому варіанті не було: без них знімок зображував би стан, неможливий за власною діаграмою класів. Далі — чотири змістові перевірки, які вдалося виконати саме на конкретних даних:

1. **Множинність 1..*` у композиції Публікація ♦ ФайлКниги` обґрунтована.****
Якби в Лабораторній роботі №2 стояло 1` до 1`, цей знімок було б неможливо побудувати: одна книга «Кобзар» фізично має і EPUB-, і PDF-версію з різними ключами об'єкта, різними розмірами й різними хешам
2. **Агрегація (а не композиція) між Колекція` і Публікація` підтверджена.** Книга «Тигрови» лежить у полиці, але має власний id` і власний статус модерації та належить публічному каталогу; видалення полиці її не знищить. Якби зв'язок був композицією, об'єкт тигрови` не міг би існувати поза полицею — а він існує.
3. **Відмова від класу Передплатник` підтверджена.** Книга «Тигрови» має `платна = true`, і доступ до неї визначається не типом об'єкта `читач_kaminskyi` (він завжди `Читач`), а наявністю окремого об'єкта `Підписка` зі статусом «активна». На знімку такого об'єкта немає — тож читач бачить «Тигрови» в каталозі, але не може її відкрити. Це рівно той сценарій, який у Лабораторній роботі №2 обґрунтував вилучення класу `Передплатник`.
4. **Виявлено дублювання атрибута `автор`.** Знімок показав те, чого на рівні типів не видно: значення `автор = "Тарас Шевченко"` стоїть одночасно в об'єкті `кобзар` : Книга` і в об'єкті `метадані_кобзар` : Метадані`, і те саме повторюється для

«Тигроловів». У Лабораторній роботі №2 атрибут `автор` справді оголошено двічі — у класі `Книга` і в класі `Метадані`. На діаграмі класів це виглядало нешкідливо, а на знімку два однакові значення в сусідніх прямокутниках одразу показують ризик розходження даних. Рішення зафіксовано тут, а не внесено заднім числом у ЛР2: у курсовій роботі атрибут слід залишити лише в `Метадані`, а `Книга.автор` зробити похідним. Це рівно той випадок, заради якого розділ 1.1 методички й радить будувати діаграму об'єктів.

Висновок: знімок із десяти об'єктів шести класів є допустимим екземпляром діаграми класів — виконано обидві обов'язкові композиції Публікація ♦ Метадані і Публікація ♦ Файлкниги; усі десять лінків мають відповідну асоціацію в Лабораторній роботі №2, конкретні значення атрибутів підтвердили правильність трьох спірних рішень попередньої роботи і виявили одне дублювання атрибута.

Розділ 3. Діаграма компонентів: модульна архітектура

3.1. Принцип декомпозиції

Методичка пропонує принцип «один компонент — одна бізнес-можливість». Його застосовано буквально: межі компонентів проведено не за технічними шарами (контролер / сервіс / репозиторій), а за доменними межами, успадкованими з пакетів діаграми класів Лабораторної роботи №2. Практичний критерій перевірки формулювався так: *чи можна описати призначення компонента одним реченням без слова «і», яке з'єднує різні бізнес-поняття?* Якщо ні — компонент треба ділити.

Усього визначено **18 компонентів**: 13 власних і 5 зовнішніх систем, а також 2 сховища даних і 17 інтерфейсів.

№	Компонент	Бізнес-можливість (одним реченням)	Походження з ЛР2
1	Веб-читалка (SPA)	Відображає каталог і текст книги в браузері та зберігає сторінку для офлайн-читання	Поняття «читалка», свідомо відкладене в ЛР2
2	API-шлюз	Приймає всі зовнішні HTTP-запити, перевіряє токен і маршрутизує їх до доменних сервісів	Технічний компонент, не має прообразу-класу
3	Сервіс акаунтів і доступу	Реєструє користувачів, автентифікує їх і зберігає ролі	`КористувачСистеми`, `Читач`, `МодераторКонтенту`, `АдміністраторСистеми`

4	Сервіс каталогу	Веде публічний перелік публікацій з пошуком і фільтрами	`Каталог`, `Публікація`, `Книга`, `Метадані`
5	Сервіс бібліотеки читача	Зберігає особисті полиці, прогрес читання, закладки й нотатки	`Колекція`, `ПрогресЧитання`, `Позначка`, `Закладка`, `Нотатка`
6	Сервіс завантаження й обробки книг	Приймає від читача файл книги, перевіряє його і кладе у сховище	`Документ`, `ФайлКниги`
7	Модуль парсерів форматів	Розбирає EPUB і PDF: витягує метадані й текст сторінки	`ІПарсерФормату`, `ПарсерEPUB`, `ПарсерPDF`
8	Сервіс підписок і платежів	Оформлює, продовжує й скасовує підписку та проводить списання	`Підписка`, `ТарифнийПлан`, `Платіж`
9	Сервіс модерації	Веде чергу перевірки завантаженого контенту й обробляє скарги	`Скарга`, `ЗавданняМодерації`
10	Сервіс сповіщень	Формує й ставить у чергу листи читачеві	`Сповіщення`
11	Модуль перекладу	Повертає переклад і словникову статтю для виділеного слова	UC11 ЛР1, не має класу в ЛР2
12	Планувальник автопродовження	За розкладом ініціює списання за підписками, у яких настав день продовження	Актор «Планувальник завдань» ЛР1
13	Сервіс журналювання та спостережуваності	Реєструє інциденти й технічні події всіх сервісів	Поняття «інцидент», свідомо відкладене в ЛР2
14	Сервіс авторизації Google OAuth	Зовнішня система: підтверджує особу користувача	Однойменний актор ЛР1
15	Об'єктне сховище файлів (S3)	Зовнішня система: зберігає й віддає файли книг	Актор «Об'єктне сховище файлів» ЛР1
16	Платіжна система	Зовнішня система: проводить списання коштів і надсилає вебхук про результат	Однойменний актор ЛР1
17	Сервіс перекладу та словник	Зовнішня система: повертає словникову	Однойменний актор ЛР1

		статтю	
18	Сервіс email-сповіщень	Зовнішня система: фізично доставляє лист	Одноіменний актор ЛР1

Два рішення тут варто пояснити окремо, бо вони не є очевидними.

Чому «Модуль парсерів форматів» — окремий компонент, а не частина сервісу завантаження. Формально обидва беруть участь в одному сценарії UC14. Але в Лабораторній роботі №2 розбір формату вже виділено в інтерфейс ІПарсерФормату з двома реалізаціями, і це поліморфна **технічна** поведінка: додавання підтримки FB2 має змінювати лише цей модуль і не чіпати логіку приймання файлів. Саме це і є означення замінного самодостатнього модуля з розділу 1.2 методички. Назву інтерфейсу збережено без змін — ІПарсерФормату, щоб трасування з ЛР2 було очевидним.

Чому «Модуль перекладу» — наш компонент, хоча переклад виконує зовнішній сервіс. Якби ми провели лінію напряду від API-шлюзу до зовнішнього словника, то сховали б власну відповідальність: нормалізацію слова, кешування результату (одне й те саме слово читачі виділяють тисячі разів) і підміну провайдера. Тому власний компонент надає ІПереклад, а зовнішній — ІСловник; це дві різні абстракції, і їх не можна зливати.

3.2. Код діаграми компонентів

Файл components.puml:

```
@startuml
' Лабораторна робота №3. Частина 2. Діаграма компонентів
' Предметна область: веб-сервіс завантаження та читання книг «Полиця»
' Камінський Олексій Дмитрович, група ВТ-23-2
' Принцип декомпозиції: один компонент — одна бізнес-можливість

left to right direction

skinparam defaultFontName "Helvetica"
skinparam shadowing false
skinparam legendBackgroundColor #FFFFFF
skinparam legendBorderColor #9AB3C6
skinparam ArrowFontSize 10
skinparam componentStyle uml2
skinparam nodesep 28
skinparam ranksep 70
skinparam component {
    BackgroundColor #FEFEFE
    BorderColor #33668E
    ArrowColor #33668E
    FontColor #000000
}
skinparam interface {
    BackgroundColor #FFFFFF
    BorderColor #33668E
}
```

```

skinparam database {
    BackgroundColor #EDF3EE
    BorderColor #4C7A54
}

' ===== КЛІЄНТСЬКИЙ РІВЕНЬ =====
component "Веб-читалка (SPA)" <<клієнт>> as WebApp #E8F0F7

' ===== СЕРВЕРНИЙ РІВЕНЬ ЗАСТОСУНКУ «ПОЛІЦЯ» =====
interface "IPублічнийAPI" as IPublicAPI
component "API-шлюз" as Gateway

interface "IAвтентифікація" as IAuth
interface "IKаталог" as ICatalog
interface "IBібліотека" as ILibrary
interface "IЗавантаження" as IIngest
interface "IPідписка" as ISubscription
interface "IMодерація" as IModeration
interface "IПереклад" as ITranslate
interface "IPлатіжнийВебхук" as IPayHook

component "Сервіс акаунтів\ні доступу" as Accounts
component "Сервіс каталогу" as Catalog
component "Сервіс бібліотеки\нчитача" as Library
component "Сервіс завантаження\нй обробки книг" as Ingest
component "Сервіс підписок\ні платежів" as Billing
component "Сервіс модерації" as Moderation
component "Модуль перекладу" as Translate
component "Планувальник\навтопродовження" as Scheduler
component "Сервіс журналювання\нта спостережуваності" as Logging

interface "IЖурнал" as ILog
interface "IPарсерФормату" as IParser
interface "ISповіднення" as INotify
component "Модуль парсерів\нформатив" as Parsers
component "Сервіс сповіщень" as Notify

' ===== ЗОВНІШНІ СИСТЕМИ =====
interface "IOAuthGoogle" as IOAuth
interface "ISховищеФайлів" as IFileStore
interface "IPлатіжнийШлюз" as IPayGw
interface "ISловник" as IDict
interface "IEmailДоставка" as IMail

component "Сервіс авторизації\нGoogle OAuth" <<зовнішня система>> as GoogleOAuth #F7EFE4
component "Об'єктне сховище\нфайлів (S3)" <<зовнішня система>> as S3 #F7EFE4
component "Платіжна система" <<зовнішня система>> as PaySystem #F7EFE4
component "Сервіс перекладу\нта словник" <<зовнішня система>> as DictAPI #F7EFE4
component "Сервіс email-сповіщень" <<зовнішня система>> as MailGw #F7EFE4

' ===== СХОВИЩА ДАНИХ =====
database "БД «Полиця»\н(PostgreSQL)" as DB
database "Кеш і черга завдань\н(Redis)" as Cache

' =====
' З'ЄДНАННЯ «ЛЬОДЯНИК-РОЗЕТКА» (ball-and-socket)
' Споживач --( Інтерфейс – необхідний інтерфейс (розетка, півколо)
' Інтерфейс - Постачальник – наданий інтерфейс (льодяник, кружечок)
' =====

WebApp --( IPublicAPI
IPublicAPI - Gateway

```

Gateway --(IAuth
IAuth - Accounts

Gateway --(ICatalog
ICatalog - Catalog

Gateway --(ILibrary
ILibrary - Library

Gateway --(IIngest
IIngest - Ingest

Gateway --(ISubscription
ISubscription - Billing

Gateway --(IModeration
IModeration - Moderation

Gateway --(ITranslate
ITranslate - Translate

Ingest --(IParser
IParser - Parsers

Accounts --(INotify
Billing --(INotify
Moderation --(INotify
INotify - Notify

Gateway --(ILog
Ingest --(ILog
Billing --(ILog
ILog - Logging

Library --(ICatalog
Library --(IFileStore
Ingest --(IModeration
Ingest --(IFileStore
Scheduler --(ISubscription

Accounts --(IOAuth
IOAuth - GoogleOAuth

IFileStore - S3

Billing --(IPayGw
IPayGw - PaySystem

' Єдиний інтерфейс, який надає НАША сторона, а потребує ЗОВНІШНЯ система:
' асинхронне підтвердження списання (вебхук POST /api/payments/callback).
IPayHook - Gateway
PaySystem --(IPayHook

Translate --(IDict
IDict - DictAPI

Notify --(IMail

IMail - MailGw

' ===== ЗАЛЕЖНОСТІ ВІД СХОВИЩ ДАНИХ (dependency) =====

Accounts ..> DB : <<use>>

Catalog ..> DB

Library ..> DB

Billing ..> DB

Moderation ..> DB

Accounts ..> Cache : <<use>>

Ingest ..> Cache : <<use>>

Notify ..> Cache

Translate ..> Cache

legend right

|= Позначення |= Зміст |

| суцільна лінія, що завершується кружечком | наданий інтерфейс (provided, «льодяник») |

| суцільна лінія, що завершується півколом навколо кружечка | необхідний інтерфейс

(required, «розетка») |

| пунктирна стрілка | залежність (dependency) |

| бежева заливка компонента | зовнішня система (поза межами розробки) |

| блакитна заливка компонента | клієнтський рівень |

| зелений циліндр | сховище даних (БД, кеш) |

endlegend

@enduml

3.3. Рисунок

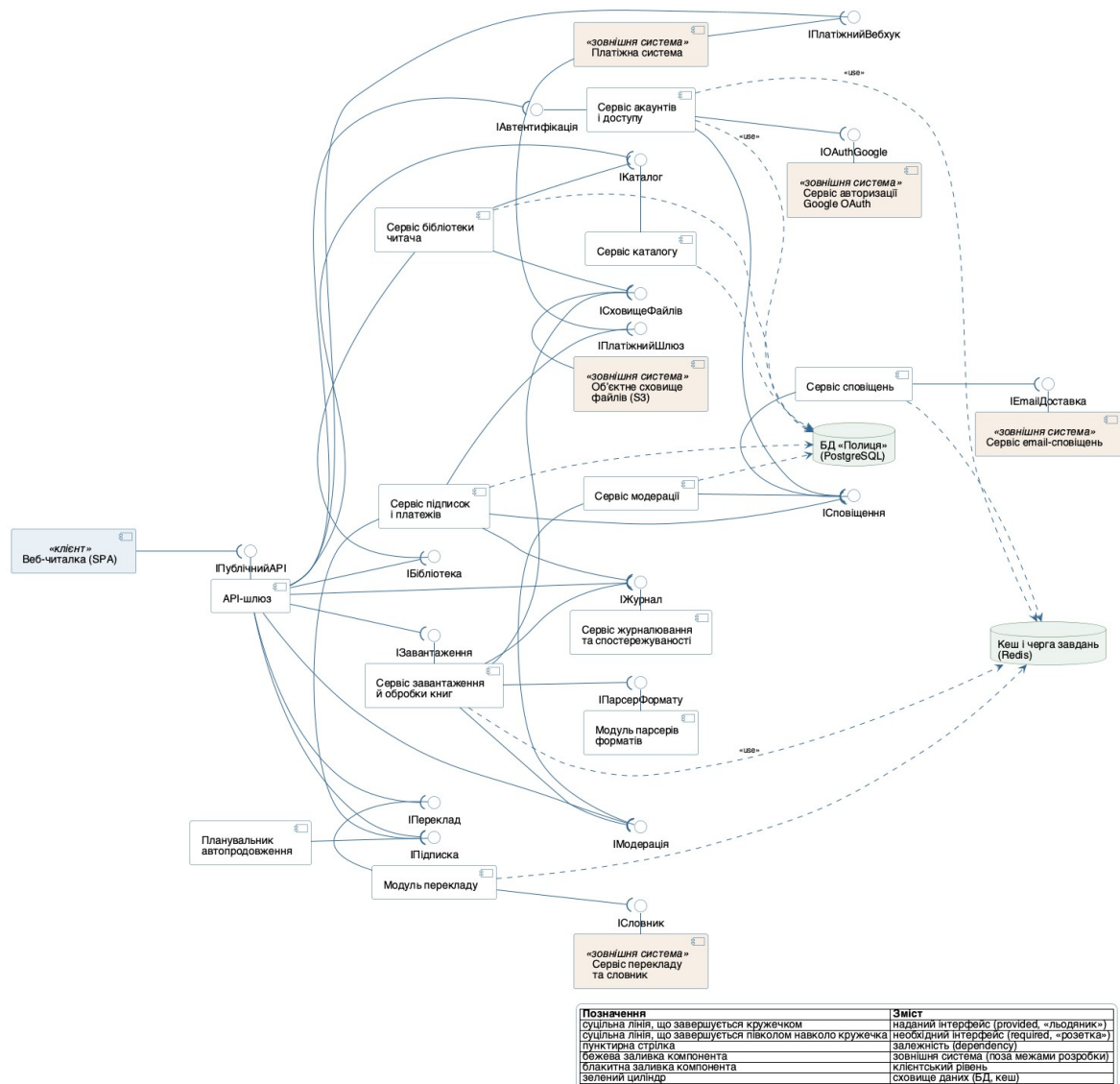


Рисунок 2 — Діаграма компонентів сервісу «Полиця»

3.4. Перелік наданих і необхідних інтерфейсів

Нотація «льодяник-розетка» (ball-and-socket) читається так: суцільна лінія до кружечка означає, що компонент **надає** інтерфейс, а лінія, яка завершується півколом навколо цього кружечка, означає, що інший компонент цей інтерфейс **потребує**. У кодї PlantUML це записано двома рядками: Інтерфейс - Постачальник і Споживач --(Інтерфейс.

Нижче повний перелік інтерфейсів кожного компонента. Прочерк означає, що компонент у цій ролі не виступає: наприклад, «Веб-читалка» нічого не надає іншим компонентам системи (її «інтерфейс» — це екран користувача), а чотири з п'яти зовнішніх систем нічого від нас не потребують.

Виняток один, і він змістовний: **платіжна система потребує наданий нами інтерфейс** ІПлатіжнийВебхук. Це асинхронне підтвердження списання, яке вона надсилає нам сама, коли банк підтвердив або відхилив транзакцію. Інтерфейс надає API-шлюз — та сама точка входу, що приймає й запитує читачів, бо вебхук фізично є звичайним HTTP-запитом ззовні й має проходити ту саму перевірку підпису й те саме журналювання. Саме цей інтерфейс робить логічну модель узгодженою з фізичною: шлях комунікації «Платіжна система — Веб-сервер» на діаграмі розгортання (розділ 4.5) має прообраз на діаграмі компонентів, а не з’являється «нізвідки». Без нього дві діаграми суперечили б одна одній.

Компонент	Надані інтерфейси (provided)	Необхідні інтерфейси (required)
Веб-читалка (SPA)	—	`ІПублічнийAPI`
API-шлюз	`ІПублічнийAPI`, `ІПлатіжнийВебхук`	`ІАвтентифікація`, `ІКаталог`, `ІБібліотека`, `ІЗавантаження`, `ІПідписка`, `ІМодерація`, `ІПереклад`, `ІЖурнал`
Сервіс акаунтів і доступу	`ІАвтентифікація`	`ІOAuthGoogle`, `ІСповіщення`
Сервіс каталогу	`ІКаталог`	—
Сервіс бібліотеки читача	`ІБібліотека`	`ІКаталог`, `ІСховищеФайлів`
Сервіс завантаження й обробки книг	`ІЗавантаження`	`ІПарсерФормату`, `ІСховищеФайлів`, `ІМодерація`, `ІЖурнал`
Модуль парсерів форматів	`ІПарсерФормату`	—
Сервіс підписок і платежів	`ІПідписка`	`ІПлатіжнийШлюз`, `ІСповіщення`, `ІЖурнал`
Сервіс модерації	`ІМодерація`	`ІСповіщення`
Сервіс сповіщень	`ІСповіщення`	`ІEmailДоставка`
Модуль перекладу	`ІПереклад`	`ІСловник`
Планувальник автопродовження	—	`ІПідписка`
Сервіс журналювання та спостережуваності	`ІЖурнал`	—
Сервіс авторизації Google OAuth *(зовнішня)*	`ІOAuthGoogle`	—
Об’єктне сховище файлів (S3) *(зовнішня)*	`ІСховищеФайлів`	—
Платіжна система *(зовнішня)*	`ІПлатіжнийШлюз`	`ІПлатіжнийВебхук`
Сервіс перекладу та словник *(зовнішня)*	`ІСловник`	—
Сервіс email-сповіщень *(зовнішня)*	`ІEmailДоставка`	—

Усього 17 інтерфейсів, кожен має рівно одного постачальника і від одного до трьох споживачів; загалом на діаграмі 17 з'єднань «надання» і 25 з'єднань «потреба». Вимогу завдання «для кожного компонента визначте хоча б один наданий та/або необхідний інтерфейс» виконано: кожен із 18 компонентів бере участь щонайменше в одному з'єднанні.

Варто звернути увагу на шість інтерфейсів, які мають більш ніж одного споживача, — саме вони доводять, що декомпозиція вдала, бо повторне використання є ознакою правильно проведеної межі:

Інтерфейс	Споживачі	Чому спільне використання виправдане
`ІСповіщення`	Сервіс акаунтів, Сервіс підписок, Сервіс модерації	Усі три надсилають листи читачеві, але жоден не має знати про SMTP
`ІЖурнал`	API-шлюз, Сервіс завантаження, Сервіс підписок	Три найризикованіші точки: зовнішній трафік, робота зі сховищем, робота з грошима
`ІСховищеФайлів`	Сервіс завантаження, Сервіс бібліотеки читача	Один кладе файл, інший формує підписане посилання на читання
`ІКаталог`	API-шлюз, Сервіс бібліотеки читача	Полиця зберігає лише ідентифікатори публікацій, а назви й обкладинки бере з каталогу
`ІПідписка`	API-шлюз, Планувальник автопродовження	Підписку оформлює людина, а продовжує таймер — обидва через один контракт
`ІМодерація`	API-шлюз, Сервіс завантаження й обробки книг	Модератор відкриває чергу перевірки через шлюз, а сервіс завантаження сам ставить у цю чергу щойно прийняту книгу

3.5. Залежності від сховищ даних

Крім з'єднань через інтерфейси, на діаграмі є **9 залежностей** (пунктирні стрілки) до двох сховищ даних. Залежність, а не інтерфейс, обрано свідомо: згідно з таблицею розділу 1.2 методички, залежність — це слабший зв'язок, який означає «зміна одного компонента може вплинути на інший». Саме так співвідносяться сервіс і його таблиці: сервіс не «викликає інтерфейс бази», він володіє своєю схемою, і зміна схеми ламає сервіс.

Сховище	Компоненти, що залежать	Що саме зберігається
БД «Полиця» (PostgreSQL)	Сервіс акаунтів, Сервіс каталогу, Сервіс бібліотеки	Персистентний стан класів LP2: акаунти, каталог, полиці,

	читача, Сервіс підписок і платежів, Сервіс модерації	прогрес, підписки, черга модерації
Кеш і черга завдань (Redis)	Сервіс акаунтів і доступу, Сервіс завантаження, Сервіс сповіщень, Модуль перекладу	Сесії й токени доступу (`sessions`), черга обробки завантажених книг (`ingest-queue`), черга листів читачам (`notify-queue`), кеш словникових статей (`dict-cache`)

Ці чотири залежності від Redis не є декоративними: кожній з них відповідає **рівно один** артефакт на вузлі Redis (розділ 4.3), і навпаки — артефакта без компонента-власника на цьому вузлі немає. Саме тому «Сервіс акаунтів і доступу» залежить не лише від бази даних, а й від кешу: він єдиний видає сесії й токени доступу, тобто він і є власником артефакта `sessions`. У першому варіанті моделі цієї залежності бракувало, і артефакт `sessions` на діаграмі розгортання висів без власника — розрив у трасуванні «компонент → артефакт → вузол», який ламав саме ту таблицю, що її розділ 4.4 називає найважливішою.

Висновок: систему розкладено на 18 компонентів за принципом однієї бізнес-можливості, що значно перевищує мінімальну вимогу завдання (5 компонентів); визначено 17 інтерфейсів, усі з'єднання виконано нотацією «льодяник-розетка», а зв'язки зі сховищами даних свідомо оформлено слабшими залежностями.

Розділ 4. Діаграма розгортання: фізична архітектура

4.1. Код діаграми розгортання

Файл `deployment.puml`:

```
@startuml
' Лабораторна робота №3. Частина 3. Діаграма розгортання
' Предметна область: веб-сервіс завантаження та читання книг «Полиця»
' Камінський Олексій Дмитрович, група ВТ-23-2

skinparam defaultFontName "Helvetica"
skinparam shadowing false
skinparam ArrowFontSize 10
skinparam nodesep 25
skinparam ranksep 45
skinparam node {
    BackgroundColor #FEFEFE
    BorderColor #33668E
    ArrowColor #33668E
}
skinparam artifact {
    BackgroundColor #F7FAFC
    BorderColor #9AB3C6
}
skinparam legendBackgroundColor #FFFFFF
skinparam legendBorderColor #9AB3C6
```

```

' ===== КЛІЄНТСЬКИЙ ВУЗОЛ =====
node "Пристрій читача\n(ПК, смартфон, планшет)" <<device>> as ClientDevice #E8F0F7 {
  node "Браузер (Chrome 141 / Safari 19)" <<execution environment>> as Browser {
    artifact "polytsia-reader.bundle.js" as ArtReader
    artifact "sw-offline.js\n(service worker)" as ArtSW
    artifact "reader-cache\n(IndexedDB)" as ArtIDB
  }
}

' ===== ЕДЖ ТА ВЕБ-СЕРВЕР =====
node "Edge-вузол CDN" <<device>> as CdnNode {
  artifact "static/\n(index.html, *.css, шрифти)" as ArtStatic
}

node "Веб-сервер\n(Nginx 1.27, Ubuntu 24.04 LTS)" <<device>> as WebNode {
  artifact "nginx.conf\n(reverse proxy, TLS termination)" as ArtNginx
  artifact "fullchain.pem\n(TLS-сертифікат)" as ArtCert
}

' ===== СЕРВЕР ЗАСТОСУНКІВ =====
node "Сервер застосунків\n(Docker-хост, Ubuntu 24.04 LTS)" <<device>> as AppNode {

  node "Контейнер polytsia-gateway" <<container>> as CtGateway {
    artifact "polytsia-gateway.jar" as ArtGateway
    artifact "application.yml" as ArtConfig
  }

  node "Контейнер polytsia-core" <<container>> as CtCore {
    artifact "accounts-service.jar" as ArtAccounts
    artifact "catalog-service.jar" as ArtCatalog
    artifact "library-service.jar" as ArtLibrary
    artifact "billing-service.jar" as ArtBilling
    artifact "moderation-service.jar" as ArtModeration
    artifact "translate-service.jar" as ArtTranslate
  }

  node "Контейнер polytsia-ingest" <<container>> as CtIngest {
    artifact "ingest-worker.jar" as ArtIngest
    artifact "format-parsers.jar\n(EPUB + PDF)" as ArtParsers
  }

  node "Контейнер polytsia-jobs" <<container>> as CtJobs {
    artifact "notify-worker.jar" as ArtNotify
    artifact "scheduler.jar\n(автопродовження підписки)" as ArtScheduler
    artifact "logging-agent.jar" as ArtLogging
    artifact "incidents.log" as ArtIncidents
  }
}

' ===== СХОВИЩА ДАНИХ =====
node "Сервер бази даних\n(PostgreSQL 16)" <<device>> as DbNode #EDF3EE {
  artifact "polytsia_db\n(схема користувачів, каталогу, \nпідписок, модерації)" as ArtDb
  artifact "schema_v12.sql" as ArtSchema
}

node "Сервер кешу та черги\n(Redis 7)" <<device>> as CacheNode #EDF3EE {
  artifact "sessions\n(сесії та токени доступу)" as ArtSessions
  artifact "ingest-queue\n(черга обробки книг)" as ArtQueue
  artifact "notify-queue\n(черга листів читачам)" as ArtNotifyQueue
  artifact "dict-cache\n(кеш словникових статей)" as ArtDictCache
}

```

```

' ===== ЗОВНІШНІ СИСТЕМИ =====
node "Об'єктне сховище файлів\n(S3-сумісне)" <<зовнішня система>> as S3Node #F7EFE4 {
    artifact "bucket polytsia-books/\n{uuid}/*.epub, *.pdf" as ArtBucket
}

node "Платіжна система" <<зовнішня система>> as PayNode #F7EFE4 {
    artifact "payments-api v3" as ArtPayApi
}

node "Сервіс авторизації\nGoogle OAuth" <<зовнішня система>> as GoogleNode #F7EFE4 {
    artifact "oauth2/v2 endpoint" as ArtOAuthApi
}

node "Сервіс перекладу\nta словник" <<зовнішня система>> as DictNode #F7EFE4 {
    artifact "dictionary-api v1" as ArtDictApi
}

node "Сервіс email-сповіщень" <<зовнішня система>> as MailNode #F7EFE4 {
    artifact "smtp-relay" as ArtSmtP
}

' ===== ШЛЯХИ КОМУНІКАЦІЇ З ПРОТОКОЛАМИ =====
ClientDevice -down- CdnNode : "HTTPS / TLS 1.3, HTTP/2\n(статика)"
ClientDevice -down- WebNode : "HTTPS / TLS 1.3,\nREST/JSON (443)"
ClientDevice -- S3Node : "HTTPS, підписане посилання S3\n(GET, дійсне 15 хв)"
CdnNode -- WebNode : "HTTPS, origin pull\n(наповнення кешу статика)"

WebNode -down- AppNode : "HTTP/1.1, 10.0.0.12:8080\n(reverse proxy, приватна мережа)"
PayNode -- WebNode : "HTTPS webhook\n(POST /api/payments/callback)"

AppNode -down- DbNode : "TCP/5432, PostgreSQL\nwire protocol + TLS"
AppNode -down- CacheNode : "TCP/6379,\nRESP + TLS"
AppNode -down- S3Node : "HTTPS, S3 API\n(PUT / GET, 443)"
AppNode -down- PayNode : "HTTPS,\nREST/JSON (443)"
AppNode -down- GoogleNode : "HTTPS, OpenID Connect\n/ OAuth 2.0 (443)"
AppNode -down- DictNode : "HTTPS,\nREST/JSON (443)"
AppNode -down- MailNode : "SMTP, TCP/587\n(STARTTLS)"

' ===== ПРИХОВАНІ ЗВ'ЯЗКИ ДЛЯ КОМПАКТНОЇ РОЗКЛАДКИ =====
' (не є елементами UML – лише підказки розкладки для Graphviz).
' Підказки між вузлами задаються через артефакти: Graphviz ранжує
' вузли всередині різних кластерів, але не самі кластери.
ArtAccounts -[hidden]down- ArtBilling
ArtCatalog -[hidden]down- ArtModeration
ArtLibrary -[hidden]down- ArtTranslate
CtGateway -[hidden]down- CtIngest
CtCore -[hidden]down- CtJobs

ArtLogging -[hidden]right- ArtIncidents
ArtSessions -[hidden]right- ArtQueue
ArtQueue -[hidden]right- ArtNotifyQueue
ArtNotifyQueue -[hidden]right- ArtDictCache

ArtReader -[hidden]down- ArtStatic
ArtReader -[hidden]down- ArtNginx
ArtNginx -[hidden]down- ArtGateway
ArtScheduler -[hidden]down- ArtDb
ArtScheduler -[hidden]down- ArtSessions
ArtScheduler -[hidden]down- ArtBucket
ArtScheduler -[hidden]down- ArtPayApi
ArtDb -[hidden]down- ArtOAuthApi

```

```
ArtSessions -[hidden]down- ArtDictApi
ArtBucket -[hidden]down- ArtSmtп

legend right
|= Позначення |= Зміст |
| <<device>> | фізичний або віртуальний обчислювальний вузол |
| <<container>> | Docker-контейнер, вкладений у вузол |
| <<зовнішня система>> | вузол поза межами нашої розробки |
| прямокутник усередині вузла | артефакт – конкретний розгорнутий файл чи образ |
| підпис на лінії | шлях комунікації із зазначенням протоколу і порту |
endlegend
@enduml
```

4.2. Рисунок

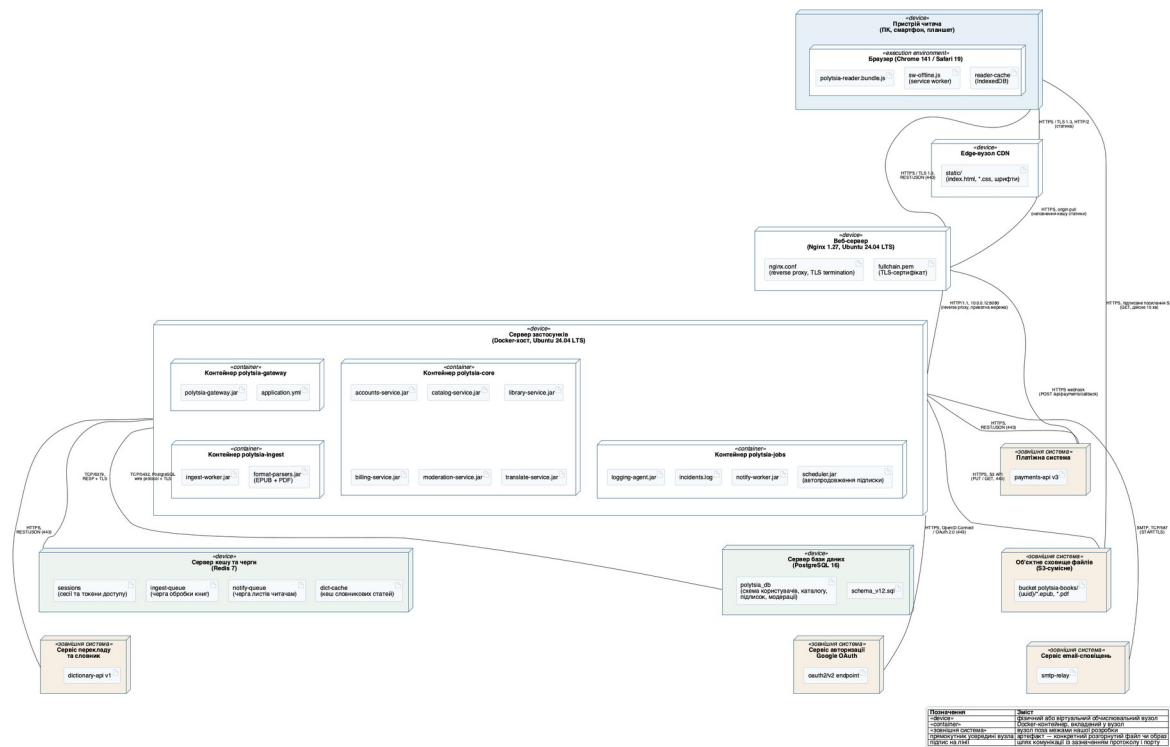


Рисунок 3 — Діаграма розгортання сервісу «Полиця»

4.3. Вузли та розміщені на них артефакти

Усього на діаграмі **16 вузлів**: 11 верхнього рівня (з них 6 наших і 5 зовнішніх) та 5 вкладених — одне середовище виконання й чотири Docker-контейнери. На них розміщено **31 артефакт**.

Вкладеність використано у двох місцях, і обидва рази вона несе зміст, а не є прикрасою: браузер — це середовище виконання всередині фізичного пристрою читача, а контейнери — ізольовані процеси всередині одного Docker-хоста.

№	Вузол	Стереотип	Артефакти на вузлі
1	Пристрій читача (ПК, смартфон, планшет)	`<<device>>`	— (містить вкладений вузол)

1.1	└ Браузер (Chrome 141 / Safari 19)	`<<execution environment>>`	`polytsia-reader.bundle.js`, `sw-offline.js`, `reader-cache (IndexedDB)`
2	Edge-вузол CDN	`<<device>>`	`static/ (index.html, *.css, шрифти)`
3	Веб-сервер (Nginx 1.27, Ubuntu 24.04 LTS)	`<<device>>`	`nginx.conf`, `fullchain.pem`
4	Сервер застосунків (Docker-хост, Ubuntu 24.04 LTS)	`<<device>>`	— (містить чотири контейнери)
4.1	└ Контейнер polytsia-gateway	`<<container>>`	`polytsia-gateway.jar`, `application.yml`
4.2	└ Контейнер polytsia-core	`<<container>>`	`accounts-service.jar`, `catalog-service.jar`, `library-service.jar`, `billing-service.jar`, `moderation-service.jar`, `translate-service.jar`
4.3	└ Контейнер polytsia-ingest	`<<container>>`	`ingest-worker.jar`, `format-parsers.jar`
4.4	└ Контейнер polytsia-jobs	`<<container>>`	`notify-worker.jar`, `scheduler.jar`, `logging-agent.jar`, `incidents.log`
5	Сервер бази даних (PostgreSQL 16)	`<<device>>`	`polytsia_db`, `schema_v12.sql`
6	Сервер кешу та черги (Redis 7)	`<<device>>`	`sessions`, `ingest-queue`, `notify-queue`, `dict-cache`
7	Об'єктне сховище файлів (S3-сумісне)	`<<зовнішня система>>`	`bucket polytsia-books/{uuid}/*.epub, *.pdf`
8	Платіжна система	`<<зовнішня система>>`	`payments-api v3`
9	Сервіс авторизації Google OAuth	`<<зовнішня система>>`	`oauth2/v2 endpoint`
10	Сервіс перекладу та словник	`<<зовнішня система>>`	`dictionary-api v1`
11	Сервіс email-сповіщень	`<<зовнішня система>>`	`smtp-relay`

Два місця в цій таблиці варто пояснити, бо вони змінилися після перевірки моделі на узгодженість.

Чому `logging-agent.jar` та `incidents.log` — це два прямокутники, а не один. Спершу вони були злиті в один артефакт із підписом у два рядки, і це була помилка нотації: це різні артефакти з різним життєвим циклом. `.jar` потрапляє на вузол під час розгортання і змінюється лише разом із версією сервісу; `incidents.log` створюється під час роботи, зростає щогодини і до розгортання взагалі не існує. Злиття до того ж спотворювало підрахунок артефактів.

Чому на вузлі Redis чотири артефакти, а не два. Таблиця розділу 3.5 обіцяє, що в Redis лежать сесії, черга обробки книг, черга листів і кеш словникових статей, а намальовано було лише перші дві. Решту додано (`notify-queue`, `dict-cache`), бо інакше компоненти «Сервіс сповіщень» і «Модуль перекладу» залежали б від вузла, на якому немає нічого їхнього.

4.4. Трасування «компонент → артефакт → вузол»

Це найважливіша таблиця розділу, бо вона реалізує вимогу завдання «розмістіть на вузлах артефакти — конкретні файли або образи, що реалізують компоненти з Частини 2» і водночас показує різницю між компонентом і артефактом із розділу 1.3 методички: компонент — логічна одиниця архітектури, артефакт — фізичний файл, що її втілює.

Компонент (розділ 3)	Артефакт	Вузол розгортання
Веб-читалка (SPA)	<code>`polytsia-reader.bundle.js`, `sw-offline.js`</code>	Браузер на пристрої читача
Веб-читалка (SPA)	<code>`static/ (index.html, *.css, шрифти)`</code>	Edge-вузол CDN
API-шлюз	<code>`polytsia-gateway.jar`, `application.yml`</code>	Контейнер <code>polytsia-gateway</code>
Сервіс акаунтів і доступу	<code>`accounts-service.jar`</code>	Контейнер <code>polytsia-core</code>
Сервіс каталогу	<code>`catalog-service.jar`</code>	Контейнер <code>polytsia-core</code>
Сервіс бібліотеки читача	<code>`library-service.jar`</code>	Контейнер <code>polytsia-core</code>
Сервіс підписок і платежів	<code>`billing-service.jar`</code>	Контейнер <code>polytsia-core</code>
Сервіс модерації	<code>`moderation-service.jar`</code>	Контейнер <code>polytsia-core</code>
Модуль перекладу	<code>`translate-service.jar`</code>	Контейнер <code>polytsia-core</code>
Сервіс завантаження й обробки книг	<code>`ingest-worker.jar`</code>	Контейнер <code>polytsia-ingest</code>
Модуль парсерів форматів	<code>`format-parsers.jar`</code>	Контейнер <code>polytsia-ingest</code>
Сервіс сповіщень	<code>`notify-worker.jar`</code>	Контейнер <code>polytsia-jobs</code>
Планувальник автопродовження	<code>`scheduler.jar`</code>	Контейнер <code>polytsia-jobs</code>
Сервіс журналювання та спостережуваності	<code>`logging-agent.jar`</code>	Контейнер <code>polytsia-jobs</code>
Об'єктне сховище файлів (S3)	<code>`bucket polytsia-books/`</code>	Об'єктне сховище (зовнішнє)
Платіжна система	<code>`payments-api v3`</code>	Платіжна система (зовнішня)

Сервіс авторизації Google OAuth	`oauth2/v2 endpoint`	Сервіс авторизації Google OAuth (зовнішній)
Сервіс перекладу та словник	`dictionary-api v1`	Сервіс перекладу (зовнішній)
Сервіс email-сповіщень	`smtp-relay`	Сервіс email-сповіщень (зовнішній)

Три випадки в цій таблиці демонструють, що зв'язок «компонент — артефакт» не є «один до одного», і це відповідь на питання 5 для самоконтролю.

- Один компонент — три артефакти на двох різних вузлах.** «Веб-читалка» розгортається як `polytsia-reader.bundle.js` і `sw-offline.js` у браузері та як `static/` на Edge-вузлі CDN. Це один логічний компонент, фізично розірваний між двома вузлами з міркувань швидкодії.
- Один компонент — два артефакти на одному вузлі.** «API-шлюз» — це `polytsia-gateway.jar` плюс `application.yml`: без конфігурації з маршрутами виконуваний файл не реалізує компонент.
- Артефакти без компонента.** `nginx.conf`, `fullchain.pem`, `polytsia\_db`, `schema\_v12.sql`, `sessions`, `ingest-queue`, `notify-queue`, `dict-cache`, `reader-cache` (IndexedDB) та `incidents.log` не реалізують жодного компонента розділу 3. Це чесно визнана інфраструктура й дані: TLS-термінація, схема бази, чотири структури в Redis, офлайн-кеш браузера і журнал інцидентів. Важливо не плутати «артефакт без компонента» з «артефактом без власника»: кожна з чотирьох структур у Redis має компонент, який її створює й читає (розділ 3.5), і `incidents.log` пише «Сервіс журналювання» — просто ці файли не є **реалізацією** своїх компонентів, реалізацією є `jar`. Приховувати цю асиметрію було б помилкою — вона є в будь-якій реальній системі.

4.5. Шляхи комунікації та протоколи

Визначено **13 шляхів комунікації**, кожен підписано протоколом і, де це має значення, портом. Підпис на кшталт просто «HTTP» вважався недостатнім: він не дає змоги ані налаштувати мережевий екран, ані оцінити, чи шифрується трафік.

Одразу слід зафіксувати особливість нотації, яку легко прийняти за недогляд. Шлях комунікації в UML є **ненапрямленою** асоціацією між вузлами — він описує наявність каналу, а не того, хто першим надсилає пакет. Тому на рисунку 3 усі тринадцять ліній намальовано без стрілок. Напрямок, у якому встановлюється з'єднання, передано не графікою, а підписом (POST /api/payments/callback, origin pull) і порядком колонок «Від вузла / До вузла» в таблиці нижче.

№	Від вузла	До вузла	Протокол і порт
1	Пристрій читача	Edge-вузол CDN	HTTPS / TLS 1.3,

			HTTP/2 (статика)
2	Пристрій читача	Веб-сервер	HTTPS / TLS 1.3, REST/JSON (443)
3	Пристрій читача	Об'єктне сховище файлів	HTTPS, підписане посилання S3 (GET, дійсне 15 хв)
4	Edge-вузол CDN	Веб-сервер	HTTPS, origin pull (наповнення кешу статика)
5	Веб-сервер	Сервер застосунків	HTTP/1.1, 10.0.0.12:8080 (reverse проху, приватна мережа)
6	Платіжна система	Веб-сервер	HTTPS webhook (POST /api/payments/callback)
7	Сервер застосунків	Сервер бази даних	TCP/5432, PostgreSQL wire protocol + TLS
8	Сервер застосунків	Сервер кешу та черги	TCP/6379, RESP + TLS
9	Сервер застосунків	Об'єктне сховище файлів	HTTPS, S3 API (PUT / GET, 443)
10	Сервер застосунків	Платіжна система	HTTPS, REST/JSON (443)
11	Сервер застосунків	Сервіс авторизації Google OAuth	HTTPS, OpenID Connect / OAuth 2.0 (443)
12	Сервер застосунків	Сервіс перекладу та словник	HTTPS, REST/JSON (443)
13	Сервер застосунків	Сервіс email-сповіщень	SMTP, TCP/587 (STARTTLS)

Чотири шляхи заслуговують окремого коментаря, бо вони неочевидні й саме їх найчастіше пропускають або підписують неправильно.

Шлях 3: пристрій читача звертається до сховища напряму. Інтуїтивно хочеться провести файл книги через сервер застосунків, але 12-мегабайтний PDF, прокачаний через власний сервер для кожного читача, з'їсть увесь канал. Тому сервер застосунків лише формує підписане посилання з обмеженим часом дії (метод `ФайлКниги.сформуваПідписанеПосилання(хвилин)` з Лабораторної роботи №2), а сам файл браузер завантажує зі сховища напряму.

Шлях 4: CDN сам забирає статика з нашого веб-сервера. У першому варіанті діаграми Edge-вузол CDN був зв'язаний лише з пристроєм читача, і це була

змістовна помилка: артефакт `static/` не мав жодного способу потрапити на CDN у межах намальованої моделі — ані `origin pull` від веб-сервера, ані деплою з сервера застосунків. Насправді працює схема `origin pull`: читач просить файл у найближчого edge-вузла; якщо файлу в кеші немає, edge-вузол забирає його з нашого веб-сервера (`origin`) і кешує. Тому шлях «Edge-вузол CDN — Веб-сервер» є таким самим реальним елементом інфраструктури, як і решта, і без нього діаграма була б неповною.

Шлях 5: між веб-сервером і сервером застосунків адреса приватна, а не `localhost`. Спершу цей шлях було підписано `127.0.0.1:8080`, і це фізично неможливо: `127.0.0.1` — це `localhost`, тобто той самий хост. Якщо Nginx проксує на `127.0.0.1`, то сервер застосунків стоїть на тій самій машині, і окремого вузла з шляхом комунікації між ними бути не може — діаграма суперечила б сама собі. Оскільки в цій моделі Nginx і Docker-хост свідомо рознесено на два вузли (щоб TLS-термінація й робоче навантаження масштабувалися окремо), підпис виправлено на адресу у внутрішній мережі: `10.0.0.12:8080`. Трафік усередині приватної мережі навмисно не шифрується — це економить процесорний час, і саме тому в підписі немає TLS, на відміну від усіх шляхів, що виходять назовні.

Шлях 6: платіжна система звертається до нас, а не навпаки. Усі інші зв'язки із зовнішніми системами ініціює наш сервер. Але результат списання приходить асинхронно — платіжна система сама викликає наш `callback`. Це рівно те поняття «вебхук», яке в Лабораторній роботі №2 було свідомо залишено поза моделлю класів із позначкою «технічний, а не доменний елемент; з'явиться в ЛРЗ». Тут воно й з'явилося — не як клас, а відразу у двох проєкціях: на діаграмі компонентів як інтерфейс `IPлатіжнийВебхук`, який надає наш API-шлюз і потребує зовнішня платіжна система (розділ 3.4), і на діаграмі розгортання як окремий шлях комунікації з підписом `POST /api/payments/callback`. Стрілки на цьому шляху немає і за нотацією бути не може; те, що ініціатива належить зовнішній стороні, видно з підпису шляху і — однозначно — з напрямку «розетки» на діаграмі компонентів.

Висновок: діаграма містить 16 вузлів (при мінімальній вимозі 4), 31 артефакт і 13 підписаних протоколом шляхів комунікації; кожен компонент розділу 3 має щонайменше один артефакт, кожен артефакт на вузлі Redis має компонент-власник, а артефакти, що не реалізують жодного компонента, чесно позначені як інфраструктура й дані.

Розділ 5. Перевірка узгодженості з Лабораторними роботами №1 і №2

Узгодженість робіт між собою є прямою вимогою методичок, тому перевірку виконано у трьох незалежних зрізах, як і в Лабораторній роботі №2.

5.1. Відображення всіх 11 акторів ЛР1 на елементи ЛР3

Перед таблицею слід назвати принцип відображення, інакше таблиці 5.1 і 5.2 виглядатимуть суперечливими. Таблиця 5.1 відображає **актора** на компонент, що реалізує його **варіанти використання**; таблиця 5.2 відображає **клас** на компонент, що володіє його **даними**. Ці два відображення не зобов'язані збігатися, і для двох акторів вони справді не збігаються: адміністратор системи як обліковий запис живе в «Сервісі акаунтів і доступу» (там його клас, його роль і його двофакторна автентифікація), а працює там, де його сценарії, — у «Сервісі підписок і платежів» (UC25 «Керувати тарифними планами»). Те саме з модератором. Щоб не плутати читача, у клітинках нижче вказано обидва компоненти з позначенням ролі кожного.

Актор ЛР1	Відображення в ЛР3
Гість	Компонент «Веб-читалка (SPA)» + анонімні маршрути `ПублічнийAPI`; вузол «Пристрій читача»
Зареєстрований читач	Той самий клієнтський компонент після проходження `ІАвтентифікація`; сесію видає «Сервіс акаунтів і доступу» і зберігає в артефакті `sessions` на вузлі Redis
Передплатник	Свідомо не окремий елемент — як і в ЛР2. Перевірку виконує «Сервіс підписок і платежів» через `Підписка`
Модератор контенту	Ідентичність і роль — «Сервіс акаунтів і доступу» (клас `МодераторКонтенту`); сценарії UC21–UC23 — «Сервіс модерації», інтерфейс `ІМодерація`, артефакт `moderation-service.jar`
Адміністратор системи	Ідентичність і роль — «Сервіс акаунтів і доступу» (клас `АдміністраторСистеми`); сценарій UC25 «Керувати тарифними планами» — «Сервіс підписок і платежів»
Платіжна система	Зовнішній компонент «Платіжна система» (надає `ІПлатіжнийШлюз`, потребує наш `ІПлатіжнийВебхук`) + однойменний вузол з артефактом `payments-api v3`
Об'єктне сховище файлів	Зовнішній компонент «Об'єктне сховище файлів (S3)» (`ІСховищеФайлів`) + вузол з `bucket polytsia-books/`
Сервіс авторизації Google OAuth	Зовнішній компонент «Сервіс авторизації Google OAuth» (`IOAuthGoogle`) + однойменний вузол

	розгортання + власний посередник «Сервіс акаунтів і доступу»
Сервіс перекладу та словник	Зовнішній компонент «Сервіс перекладу та словник» (ІСловник) + власний «Модуль перекладу» як посередник
Сервіс email-сповіщень	Зовнішній компонент «Сервіс email-сповіщень» (ІEmailДоставка) + однойменний вузол + власний посередник «Сервіс сповіщень»
Планувальник завдань	Компонент «Планувальник автопродовження», артефакт `scheduler.jar` у контейнері polytsia-jobs. Межу системи тут свідомо зміщено: у ЛР1 це зовнішній актор зі стереотипом `<<таймер>>`, бо він ініціює UC20 зовні щодо ділової логіки; у ЛР3 таймер є частиною нашої поставки (ми пишемо його код і розгортаємо його образ), тому він показаний власним внутрішнім компонентом, а не зовнішньою системою

Усі 11 акторів відображені. Закономірність очевидна: кожна зовнішня система ЛР1 дала рівно один зовнішній компонент і рівно один вузол розгортання, а чотири з п'яти — ще й **єдиний** власний компонент-посередник: «Сервіс акаунтів і доступу» для Google OAuth, «Сервіс сповіщень» для поштового шлюзу, «Модуль перекладу» для словника, «Сервіс підписок і платежів» для платіжної системи. За цим стоїть правило: зовнішній API має мати в нашій системі рівно одну точку виклику, щоб заміна провайдера зачіпала рівно один компонент.

П'ята зовнішня система — об'єктне сховище файлів — є свідомим винятком із цього правила, і його слід назвати прямо, а не замовчувати: інтерфейс ІСховищеФайлів споживають **два** наші компоненти — «Сервіс завантаження й обробки книг» і «Сервіс бібліотеки читача». Посередника тут не вводили тому, що ці два компоненти виконують різні операції й не мають спільного коду: перший кладе файл у сховище (PUT під час завантаження книги), другий лише формує підписане посилання на читання (GET з обмеженим часом дії, шлях 3 розділу 4.5). Компонент-посередник, який не робив би нічого, крім передавання виклику далі, був би зайвим рівнем. Ціна рішення чесна і названа заздалегідь: при зміні провайдера сховища доведеться правити два компоненти замість одного.

5.2. Відображення всіх 24 елементів моделі класів ЛР2 на компоненти

Пакет і класи ЛР2	Компонент ЛР3
Користувачі та доступ: `КористувачСистеми`, `Читач`, `МодераторКонтенту`,	Сервіс акаунтів і доступу

`АдміністраторСистеми`	
Користувачі та доступ: `Сповідання`	Сервіс сповіщень
Контент: `Публікація`, `Книга`, `Метадані`, `Каталог`	Сервіс каталогу
Контент: `Документ`, `ФайлКниги`	Сервіс завантаження й обробки книг
Контент: `Колекція`	Сервіс бібліотеки читача
Читання: `ПрогресЧитання`, `Позначка`, `Закладка`, `Нотатка`	Сервіс бібліотеки читача
Підписка та оплата: `Підписка`, `ТарифнийПлан`, `Платіж`	Сервіс підписок і платежів
Модерація: `Скарга`, `ЗавданняМодерації`	Сервіс модерації
Обробка форматів: `ПарсерФормату`, `ПарсерEPUB`, `ПарсерPDF`	Модуль парсерів форматів

Усі 24 елементи (20 класів, 3 абстрактні, 1 інтерфейс) розподілено між вісьмома доменними компонентами; жоден клас не залишився без компонента і жоден не потрапив у два компоненти одночасно. Межі пакетів ЛР2 майже збіглися з межами компонентів — єдине розходження полягає в тому, що клас Колекція лежить у пакеті «Контент», але за змістом належить бібліотеці читача (полиця — це особиста добірка, а не частина публічного каталогу). Це розходження не є помилкою жодної з робіт: пакет діаграми класів групує за **спорідненістю понять**, а компонент — за **бізнес-можливістю**, і ці критерії не зобов'язані збігатися.

П'ять власних компонентів не мають прообразу серед класів — API-шлюз, Веб-читалка, Модуль перекладу, Планувальник автопродовження і Сервіс журналювання, — а разом із п'ятьма зовнішніми системами таких компонентів десять. Це нормально й очікувано: діаграма класів моделює предметну область, а діаграма компонентів — ще й технічну архітектуру, якої в домені немає.

5.3. Три поняття, свідомо відкладені в ЛР2, і їхнє місце тут

У Лабораторній роботі №2 три іменники з текстових специфікацій було чесно залишено поза моделлю класів із прямою вказівкою, що вони з'являться в Лабораторній роботі №3. Обіцянку виконано повністю.

Поняття	Обіцянка в ЛР2	Де воно з'явилося в ЛР3
«читалка»	«елемент інтерфейсу користувача, а не сутність зі станом»	Компонент «Веб-читалка (SPA)» та артефакти `polytsia-reader.bundle.js`, `sw-offline.js`, `reader-cache (IndexedDB)` на вузлі браузера
«вебхук»	«механізм інтеграції з платіжною системою, технічний	Інтерфейс `ІПлатіжнийВебхук` на діаграмі компонентів (надає

	елемент»	наш API-шлюз, потребує зовнішня платіжна система — єдиний випадок, коли зовнішня система щось потребує від нас) і шлях комунікації № 6 «Платіжна система — Веб-сервер, HTTPS webhook (POST /api/payments/callback)» на діаграмі розгортання
«інцидент»	«технічний артефакт спостережуваності; йому місце серед компонентів логування ЛРЗ»	Компонент «Сервіс журналювання та спостережуваності» з інтерфейсом `ІЖурнал` та два артефакти — `logging-agent.jar` (реалізація) і `incidents.log` (самі інциденти) — у контейнері polytsia-jobs

Висновок: узгодженість підтверджено у трьох зрізах — усі 11 акторів ЛР1 і всі 24 елементи моделі класів ЛР2 мають відображення в ЛР3, а всі три поняття, відкладені в ЛР2 «на потім», справді знайшли своє місце. Назви збережено буквально, і це стосується не лише доменних понять та інтерфейсу ІПарсерформату, а й усіх п'яти зовнішніх систем: компоненти й вузли ЛР3 названо «Сервіс авторизації Google OAuth», «Сервіс email-сповіщень», «Платіжна система», «Об'єктне сховище файлів», «Сервіс перекладу та словник» — тими самими словами, якими названо акторів у ЛР1. У першому варіанті роботи ці назви розходилися («Google OAuth 2.0» на діаграмі компонентів і «Google Identity» на діаграмі розгортання проти «Сервіс авторизації Google OAuth» у ЛР1), що робило трасування приблизним. Єдине допущене доповнення — технічне уточнення «(S3)» і «(S3-сумісне)» у назвах компонента й вузла сховища: воно не змінює поняття, а називає конкретний протокол. Єдина свідома зміна межі системи — часовий актор «Планувальник завдань», об'єднана в розділі 5.1.

Критичний аналіз ІІІ-чернетки

Чернетки всіх трьох діаграм згенеровано ІІІ-асистентом за текстовим описом предметної області та результатами Лабораторних робіт №1 і №2 — розділ 1.4 методички прямо описує такий сценарій для кожного з трьох типів діаграм. Нижче наведено шість реальних помилок саме цих чернеток. Кожна стосується конкретного елемента цієї предметної області, а не є загальною фразою.

Помилка ІІІ-чернетки	Чому це помилка	Як виправлено
На діаграмі об'єктів лінки підписано множинністю: <code>`читач_kaminskyi "1" -- "0..*"</code> <code>полиця_класика`</code>	Пряме порушення таблиці розділу 1.1 методички: множинність є властивістю асоціації на рівні типів, а на знімку вона вже розгорнута в конкретну кількість об'єктів. Писати «0..*» поруч із єдиним намальованим об'єктом — означає стверджувати два різні факти одночасно	Множинність прибрано з усіх десяти лінків. Натомість її показано так, як і належить на знімку: двома об'єктами <code>`ФайлКниги`</code> для «Кобзаря» і одним для «Тигроловів» (ілюстрація верхньої та нижньої меж <code>`1..*`</code>) і двома книгами в одній полиці (ілюстрація <code>`0..*`</code>)
Знімок не є допустимим екземпляром діаграми класів: створено об'єкт <code>`кобзар`</code> : Публікація, у жодної книги немає об'єкта <code>`Метадані`</code> , а в «Тигроловів» — жодного <code>`ФайлКниги`</code>	Три помилки одного роду: ІІІ будує знімок «на око», а не як екземпляр моделі. <code>`Публікація`</code> у ЛР2 оголошено абстрактним класом, а абстрактний клас не має екземплярів. Композиції <code>`Публікація "1" *-- "1" Метадані`</code> і <code>`Публікація "1" *-- "1..*" ФайлКниги`</code> є обов'язковими, тож книга без метаданих або без жодного файлу — неможливий стан системи. Знімок, що зображує неможливий стан, нічого не перевіряє, а спростовує сам себе	Замінено на <code>`кобзар : Книга`</code> , у прямокутнику показано значення успадкованих атрибутів <code>`Публікація`</code> (<code>`id`</code> , <code>`назва`</code> , <code>`мова`</code> , <code>`статусМодерації`</code>) поряд із власними атрибутами <code>`Книга`</code> . Додано три об'єкти, яких бракувало: <code>`метадані_кобзар`</code> , <code>`метадані_тигролови`</code> , <code>`тигролови_еруб`</code> . Знімок зріс із семи об'єктів до десяти, звірка розділу 2.4 — із семи лінків до десяти
Декомпозиція на компоненти за технічними шарами: «Контролери», «Сервіси», «Репозиторії», «Моделі», «Утиліти»	Формально дає потрібні п'ять компонентів, але суперечить принципу «один компонент — одна бізнес-можливість». Шар не є замінним самодостатнім модулем: не можна замінити «усі репозиторії» як одиницю й не можна виділити шар у мікросервіс. Така діаграма нічого не каже про систему — вона однакова для сервісу читання книг і для доставки їжі	Межі проведено за доменними межами, успадкованими з пакетів ЛР2: 13 власних компонентів, кожен описується одним реченням без сполучника «і». Перевірку виконано таблицею розділу 3.1
Зовнішні системи «сховані» всередині наших сервісів: клас S3 і платіжний шлюз зображені як частина сервісу завантаження й сервісу оплати відповідно	У ЛР1 це окремі актори, і вони нам не належать — ми не керуємо їхнім життєвим циклом і не можемо їх змінювати. Без окремих компонентів немає чого вести необхідні інтерфейси <code>`ІСховищеФайлів`</code> і <code>`ІПлатіжнийШлюз`</code> , а на	Виділено п'ять зовнішніх компонентів зі стереотипом <code>`<<зовнішня система>>`</code> і власною заливкою; кожен надає рівно один інтерфейс і кожному відповідає окремий вузол розгортання

	діаграмі розгортання не з'являться відповідні вузли й шляхи комунікації — тобто помилка з ЛРЗ поширюється далі	
На діаграмі розгортання у вузли поміщено прямокутники з назвами «Сервіс оплати», «Сервіс каталогу», «Модуль перекладу»	Сплутано компонент і артефакт — найтипівіша помилка саме цієї діаграми (розділ 1.3 методички). Компонент є логічною одиницею архітектури, у вузол розгортають фізичний файл чи образ. Діаграма розгортання з логічними компонентами всередині вузлів просто дублює діаграму компонентів і не додає нової інформації	Усі елементи у вузлах замінено на конкретні артефакти (`billing-service.jar`, `catalog-service.jar`, `format-parsers.jar`, `bucket polytsia-books/`) і побудовано таблицю трасування «компонент → артефакт → вузол» розділу 4.4, де окремо показано три випадки, коли відповідність не є «один до одного»
Шляхи комунікації підписано як «НТТР», «запити», «база даних»; зв'язок із платіжною системою показано лише в один бік	Підпис «НТТР» не дає ані налаштувати мережевий екран, ані зрозуміти, чи шифрується трафік; п. 3 завдання прямо вимагає зазначити протокол. Односторонній зв'язок із платіжною системою — змістовна помилка: результат списання приходить асинхронно, і без зворотного виклику підписка ніколи не стане активною	Усі 13 шляхів підписано протоколом і портом (`TCP/5432`, `TCP/6379`, `SMTP, TCP/587 (STARTTLS)`, `HTTPS / TLS 1.3`, `S3 API`, `OpenID Connect`). Додано шлях «Платіжна система — Веб-сервер: HTTPS webhook» і парний йому інтерфейс `ПлатіжнийВебхук` на діаграмі компонентів — те саме поняття «вебхук», яке в ЛР2 було відкладено саме до цієї роботи

Окремо варто зафіксувати межі корисності ШІ в цій роботі, бо вони виявилися іншими, ніж у Лабораторних роботах №1 і №2.

Де ШІ допоміг найбільше — у діаграмі об'єктів. Згенерувати десяток правдоподібних наборів тестових даних (назви книг, ISBN, розміри файлів, відсотки прочитаного, часові мітки) вручну довго й нудно, а помилитися в них майже неможливо. Це рівно той сценарій, який описує розділ 1.4 методички, і він справді працює.

Де ШІ допоміг найменше — у розкладці. Жодна з трьох згенерованих чернеток не рендерилася в читабельному вигляді: діаграма компонентів розповзалася в один ряд із накладеними підписами, діаграма розгортання впиралася в ліміт 4096 пікселів і обрізалася. ШІ генерує синтаксично коректний PlantUML, але не бачить

результату й тому не може оцінити, чи читабельна діаграма. Рішення — директива `left to right direction` для компонентів і приховані зв'язки між артефактами для розгортання — знайдено експериментально, за кілька ітерацій «відрендерив → подивився → виправив».

Де III був небезпечний — у виборі інфраструктури. Чернетка діаграми розгортання містила Kubernetes-кластер із трьох вузлів, реплікацію PostgreSQL «primary + 2 replica» і окремий вузол Elasticsearch. Усе це синтаксично бездоганно і виглядає солідно, але жодне з цих рішень не впливає з вимог: у Лабораторній роботі №1 немає жодного нефункціонального показника, який би їх обґрунтував. Це пряме підтвердження тези розділу 1.4 методички: реальні характеристики продуктивності й відмовостійкості III не перевіряє. Інфраструктуру зведено до одного Docker-хоста з чотирма контейнерами — рівно стільки, скільки потрібно навчальному проєкту, і кожен контейнер має пояснення, чому його відокремлено.

Висновок: III виявився корисним генератором тестових даних і синтаксису PlantUML, але систематично помилявся у трьох напрямках: плував рівні абстракції (екземпляр абстрактного класу, множинність на знімку, компонент замість артефакта), ігнорував зовнішні системи й пропонував надлишкову інфраструктуру, не обґрунтовану вимогами. Жодну з цих помилок неможливо виявити без знання власної предметної області та без результатів Лабораторних робіт №1 і №2.

Відповіді на питання для самоконтролю

1. Чим відрізняється лінк (link) на діаграмі об'єктів від асоціації на діаграмі класів?

Асоціація з'єднує **типи** й описує всі можливі конфігурації одразу, тому несе множинність: публікація "1" *-- "1..*" файлкини означає «у кожній публікації є один або більше файлів». Лінк з'єднує **конкретні екземпляри** в конкретний момент і множинності не має, бо вона вже розгорнута в кількість намальованих об'єктів. На рисунку 1 одній асоціації публікація – файлкини відповідають три окремі лінки: кобзар – кобзар_еруб, кобзар – кобзар_pdf і тигролови – тигролови_еруб. Одна асоціація може породити нуль, один або тисячу лінків; лінк без асоціації-прообразу є помилкою моделі.

2. Наведіть приклад ситуації, коли доцільно побудувати діаграму об'єктів для перевірки коректності діаграми класів

Саме така ситуація виникла в цій роботі з відношенням колекція – публікація. На діаграмі класів абстрактно сперечатися, агрегація це чи композиція, можна довго.

Але щойно ми спробували намалювати знімок, питання вирішилося за секунду: книга «Тигролови» лежить у полиці «Українська класика» і водночас має власний id, власний статус модерації та належить публічному каталогу. Якби зв'язок був композицією, об'єкт тигролови не міг би існувати поза полицею й мусив би знищитися разом із нею — а на знімку він очевидно самотійний. Конкретні дані зробили помилку видимою там, де абстрактна діаграма її приховувала. Загальне правило: діаграму об'єктів варто будувати щоразу, коли множинність або тип відношення викликають сумнів, а також коли структуру треба пояснити замовникові.

3. У чому різниця між наданим (provided) і необхідним (required) інтерфейсом компонента?

Наданий інтерфейс — це обіцянка компонента: набір операцій, які він гарантує виконати для інших, позначається «льодяником» (лінія з кружечком). Необхідний — це його вимога до середовища: без цього інтерфейсу компонент не запрацює, позначається «розеткою» (півколо). Один і той самий інтерфейс завжди виступає в обох ролях з різних боків: Ісповіщення є наданим для «Сервісу сповіщень» і необхідним для трьох компонентів — сервісів акаунтів, підписок і модерації. Практичне значення різниці: надані інтерфейси утворюють публічний контракт і їх не можна змінювати без узгодження зі споживачами, а необхідні показують, що саме треба підставити, щоб компонент можна було протестувати ізольовано.

4. Чим компонент відрізняється від артефакта на діаграмі розгортання?

Компонент — логічна одиниця архітектури, відповідь на питання «що система вміє»: «Сервіс підписок і платежів». Артефакт — фізичний файл, який розгортають на вузлі, відповідь на питання «що саме лежить на диску»: `billing-service.jar`. Компонент не можна скопіювати на сервер, артефакт не можна обговорювати із замовником. Практичний критерій розрізнення: якщо в назві елемента всередині вузла немає розширення файлу, версії образу чи імені каталогу — найімовірніше, туди помилково потрапив компонент. Саме цю помилку містила ШІ-чернетка діаграми розгортання.

5. Чому один компонент іноді реалізується кількома артефактами (і навпаки)?

У цій роботі є обидва випадки. Компонент «Веб-читалка (SPA)» реалізується трьома артефактами на двох різних вузлах: `polytsia-reader.bundle.js` і `sw-offline.js` у браузері плюс `static/` на CDN — логічно це один модуль, але фізично його розривають міркування швидкодії. Компонент «API-шлюз» реалізується парою `polytsia-gateway.jar` + `application.yml`, бо без конфігурації з

маршрутами виконуваний файл не робить нічого. Зворотного випадку — коли один артефакт реалізує **кілька компонентів** — на цій діаграмі чесно немає, і це теж результат: поставку свідомо побудовано так, щоб кожен компонент мав власний .jar і міг бути перерозгорнутий окремо від решти. Артефакт format-parsers.jar зворотним випадком не є, хоча на перший погляд схожий: ПарсерEPUB і ПарсерPDF — це два **класи** одного компонента «Модуль парсерів форматів», а не два компоненти. Як виглядав би справжній зворотний випадок: якби шість доменних сервісів контейнера polytsia-core збиралися не в шість .jar, а в один polytsia-core.jar, цей єдиний артефакт реалізовував би одразу шість компонентів — саме так влаштований типовий моноліт. Показово, що діаграма компонентів при такому злитті не змінилася б узагалі: межі відповідальності лишаються ті самі, змінюється лише одиниця складання. Причина асиметрії в обидва боки одна: логічні межі проводяться за відповідальністю, а фізичні — за одиницею складання й розгортання, і ці критерії незалежні.

6. Які обмеження має ШІ-згенерована декомпозиція системи на компоненти?

Їх три, і всі три проявилися в цій роботі. По-перше, ШІ не знає нефункціональних вимог і тому не може обрати «зернистість»: його чернетка пропонувала або п'ять технічних шарів, або, навпаки, Kubernetes-кластер з окремим мікросервісом на кожну сутність. По-друге, ШІ схильний до шаблонної шарової декомпозиції (контролери / сервіси / репозиторії), яка однакова для будь-якої предметної області, а отже не несе інформації про конкретну систему. По-третє, ШІ недооцінює зовнішні системи, «вбудовуючи» їх у власні сервіси, — і ця помилка поширюється далі, бо позбавляє діаграму розгортання відповідних вузлів і шляхів комунікації. Спільна причина всіх трьох: межі компонентів визначаються не текстом опису, а рішеннями про те, що має змінюватися незалежно, — а ці рішення спираються на знання про майбутній розвиток системи, якого в ШІ немає.

Висновки

У Лабораторній роботі №3 побудовано три структурні діаграми UML для веб-сервісу завантаження та читання книг «Полиця», що є предметною областю курсової роботи. Діаграма об'єктів містить десять об'єктів шести класів і десять лінків; діаграма компонентів — 18 компонентів (13 власних і 5 зовнішніх), 17 інтерфейсів, 25 з'єднань «розетка», 17 з'єднань «льодяник» і 9 залежностей від двох сховищ даних; діаграма розгортання — 16 вузлів, 31 артефакт і 13 шляхів комунікації з протоколами. Усі три перевищують мінімальні вимоги завдання (4–5 класів на знімку, 5 компонентів, 4 вузли), причому шостий клас на діаграмі

об'єктів з'явився не заради обсягу: без об'єктів метадані знімок не був би допустимим екземпляром діаграми класів (розділ 2.1). Кожен файл пройшов перевірку `plantuml -checkonly` з кодом повернення 0 і відрендерений у SVG та PNG.

Головним результатом роботи є не самі діаграми, а підтверджена наскрізна узгодженість моделі. Усі 11 акторів Лабораторної роботи №1 і всі 24 елементи моделі класів Лабораторної роботи №2 мають явне відображення в компонентах і вузлах цієї роботи, причому назви збережено буквально — інтерфейс `ІПарсерФормату` перейшов з діаграми класів на діаграму компонентів без перейменування, а всі п'ять зовнішніх систем названо в ЛР3 тими самими словами, якими їх названо акторами в ЛР1. Окремо закрито «борг» попередньої роботи: три поняття («читалка», «вебхук», «інцидент»), свідомо залишені поза моделлю класів із позначкою «з'явиться в ЛР3», справді з'явилися — відповідно як компонент із трьома артефактами на двох вузлах, як пара «наданий інтерфейс `ІПлатіжнийВебхук` + шлях комунікації з підписом `POST /api/payments/callback`» і як окремий компонент журналювання з двома артефактами. Єдина свідомо зміна межі системи — часовий актор «Планувальник завдань», який у ЛР1 був зовнішнім, а в ЛР3 став нашим внутрішнім компонентом; це розходження не замовчано, а обґрунтовано в розділі 5.1. Такий підхід перетворює чотири окремі лабораторні на одну зв'язну модель.

Робота також показала, що діаграма об'єктів є не ілюстрацією, а інструментом перевірки. Саме конкретні значення атрибутів підтвердили три спірні рішення Лабораторної роботи №2: обґрунтованість множинності `1..*` у композиції публікація ♦ файлкниги (одна книга справді має EPUB- і PDF-версії), правильність вибору агрегації замість композиції між колекцією та публікацією (книга «Тигролови» самотійна) і відмову від класу передплатник (доступ визначається наявністю об'єкта підписки, а не типом об'єкта читача). Четвертий результат виявився неприємним, але корисним: знімок показав, що атрибут автор дубльовано в класах книга і метадані — на рівні типів це непомітно, а на рівні екземплярів два однакові значення стоять у сусідніх прямокутниках. Жодну з цих перевірок неможливо виконати на рівні абстрактної діаграми класів.

Стосовно ролі ІІІ висновок виявився тоншим, ніж у попередніх роботах. ІІІ справді зекономив час на генерації правдоподібних тестових даних для діаграми об'єктів і на синтаксисі PlantUML, але систематично помилявся у плутанині рівнів абстракції: створив екземпляр абстрактного класу, поставив множинність на лінки

знімка, зібрав знімок без обов'язкових частин композицій, помістив у вузли компоненти замість артефактів і підписав шлях між двома різними вузлами loorback-адресою 127.0.0.1. Окремо показовою була схильність пропонувати надлишкову інфраструктуру (Kubernetes, реплікація БД, Elasticsearch), не обґрунтовану жодною вимогою, — рівно те обмеження, про яке попереджає розділ 1.4 методички. Крім того, ШІ не бачить результату рендерингу, тому читабельність усіх трьох діаграм довелося доводити вручну через кілька ітерацій розкладки.

На завершення — відповідь на питання, яка з трьох діаграм найкраще пояснює архітектуру системи стороннім особам. Відповідь залежить від того, хто ця стороння особа, і саме в цьому полягає цінність наявності трьох діаграм, а не однієї. **Замовникові** найзрозумілішою є діаграма об'єктів: вона оперує реальними назвами книг і конкретними числами, не потребує знання нотації UML і дозволяє одразу сказати «ні, у нас книга може бути в кількох полицях одночасно». **Новому розробникові** найкориснішою є діаграма компонентів: вона за кілька хвилин відповідає на головні питання першого дня — які є модулі, хто кого викликає, через який контракт і що буде зламано, якщо змінити сигнатуру. **Інженерові з експлуатації** потрібна діаграма розгортання: лише на ній видно, які порти відкривати, де термінується TLS і яка саме зовнішня система звертається до нас, а не ми до неї. Якщо ж обирати одну-єдину, то це діаграма компонентів: вона займає середній рівень абстракції — достатньо конкретна, щоб бути перевірюваною, і достатньо загальна, щоб залишатися правдивою після зміни інфраструктури чи набору даних.